

1480. Running Sum of 1d Array

Easy Topics Companies Hint

Given an array `nums`. We define a running sum of an array as `runningSum[i] = sum(nums[0]..nums[i])`.

Return the running sum of `nums`.

Example 1:

Input: `nums = [1,2,3,4]`
Output: `[1,3,6,10]`
Explanation: Running sum is obtained as follows: `[1, 1+2, 1+2+3, 1+2+3+4]`.

Example 2:

Input: `nums = [1,1,1,1,1]`
Output: `[1,2,3,4,5]`
Explanation: Running sum is obtained as follows: `[1, 1+1, 1+1+1, 1+1+1+1, 1+1+1+1+1]`.

Example 3:

Input: `nums = [3,1,2,10,1]`
Output: `[3,4,6,16,17]`

Constraints:

- $1 \leq \text{nums.length} \leq 1000$
- $-10^6 \leq \text{nums}[i] \leq 10^6$



Solved

Code Accepted

All Submissions

Runtime

0 ms Beats 100.00%

Memory

44.45 MB Beats 17.33%



Code | Java

```
1 class Solution {
2     public int[] runningSum(int[] nums) {
3         for (int i = 1; i < nums.length; i++) {
4             nums[i] += nums[i - 1];
5         }
6         return nums;
7     }
8 }
```

Write your notes here

Line: 51 Col: 1

 Upload Code as File

☐ Test against custom input

Run Code

Submit Code



You have earned 20.00 points!
You are now 5 points away from the 1st star for your java badge.

80%

20/25

Congratulations

You solved this challenge. Would you like to challenge your friends?  

Next Challenge

✓ Test case 0

✓ Test case 1 

✓ Test case 2 

✓ Test case 3 

✓ Test case 4 

✓ Test case 5 

✓ Test case 6 

Compiler Message

Success

Input (stdin)

```
1 6 3
2 5 3 5 2 3 2
```

[Download](#)

Expected Output

```
1 3
```

[Download](#)

Line: 24 Col: 10

 Upload Code as File

☐ Test against custom input

Run Code

Submit Code

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Input (stdin)

Download

```
1 5
2 john tom
3 john mary
4 john tom
5 mary anna
6 mary anna
```

Your Output (stdout)

```
1 1
2 2
3 2
```

← → ↺

leetcode.com/problems/remove-element/

☆

🏠 Problem List < > 🔍

🔥 Submit 📄 🚀

🔧 0 🕒 👤

Description Editorial Solutions Submissions

<

27. Remove Element

Easy Topics Companies Hint

Given an integer array `nums` and an integer `val`, remove all occurrences of `val` in `nums` **in-place**. The order of the elements may be changed. Then return *the number of elements in `nums` which are not equal to `val`*.

Consider the number of elements in `nums` which are not equal to `val` be `k`, to get accepted, you need to do the following things:

- Change the array `nums` such that the first `k` elements of `nums` contain the elements which are not equal to `val`. The remaining elements of `nums` are not important as well as the size of `nums`.
- Return `k`.

Custom Judge:

The judge will test your solution with the following code:

```
int[] nums = [...]; // Input array
int val = ...; // Value to remove
int[] expectedNums = [...]; // The expected answer with correct length.
// It is sorted with no values equaling val.

int k = removeElement(nums, val); // Calls your implementation

assert k == expectedNums.length;
sort(nums, 0, k); // Sort the first k elements of nums
for (int i = 0; i < actualLength; i++) {
    assert nums[i] == expectedNums[i];
}
```

👍 5.3K 🗨 984 ☆ 📄 ?

</> Code

Java 🔒 Auto

```
1 class Solution {
2     public int removeElement(int[] nums, int val) {
3         int k = 0;
4
5         for (int i = 0; i < nums.length; i++) {
6             if (nums[i] != val) {
7                 nums[k] = nums[i];
8                 k++;
9             }
10        }
11    }
```

Saved

✅ Testcase ➤ Test Result

Accepted Runtime: 0 ms

✅ Case 1 ✅ Case 2

Input

nums =
[3,2,2,3]

val =
3

Output

Problem List

Accepted

Editorial

Solutions

Submissions

26. Remove Duplicates from Sorted Array

EasyTopicsCompaniesHint

Given an integer array `nums` sorted in **non-decreasing order**, remove the duplicates **in-place** such that each unique element appears only **once**. The **relative order** of the elements should be kept the **same**.

Consider the number of *unique elements* in `nums` to be `k`. After removing duplicates, return the number of unique elements `k`.

The first `k` elements of `nums` should contain the unique numbers in **sorted order**. The remaining elements beyond index `k - 1` can be ignored.

Custom Judge:

The judge will test your solution with the following code:

```
int[] nums = [...]; // Input array
int[] expectedNums = [...]; // The expected answer with correct length

int k = removeDuplicates(nums); // Calls your implementation

assert k == expectedNums.length;
for (int i = 0; i < k; i++) {
    assert nums[i] == expectedNums[i];
}
```

If all assertions pass, then your solution will be **accepted**.

Solved

Code

JavaAuto

```
1 class Solution {
2     public int removeDuplicates(int[] nums) {
3         if (nums.length == 0) {
4             return 0;
5         }
6
7         // Pointer for tracking the position of unique elements
8         int k = 1;
9
10        for (int i = 1; i < nums.length; i++) {
11            // Found a new unique element
```

TestcaseTest Result

AcceptedRuntime: 0 ms

Case 1Case 2

Input

nums =
[1,1,2]

Output

[1,2]

53. Maximum Subarray

Medium Topics Companies

Given an integer array `nums`, find the **subarray** with the largest sum, and return *its* sum.

Example 1:

Input: `nums = [-2,1,-3,4,-1,2,1,-5,4]`

Output: 6

Explanation: The subarray `[4,-1,2,1]` has the largest sum 6.

Example 2:

Input: `nums = [1]`

Output: 1

Explanation: The subarray `[1]` has the largest sum 1.

Example 3:

Input: `nums = [5,4,-1,7,8]`

Output: 23

Explanation: The subarray `[5,4,-1,7,8]` has the largest sum 23.

Constraints:

Solved ✓

Java Auto

⋮ 🔖 ⌂ ↺

```
1 class Solution {
2     public int maxSubArray(int[] nums) {
3         int maxSoFar = nums[0];
4         int currentMax = nums[0];
5
6         for (int i = 1; i < nums.length; i++) {
7             // Decide whether to add current element to existing subarray or start a new
              subarray
8             currentMax = Math.max(nums[i], currentMax + nums[i]);
9             // Track the highest sum seen so far
10            maxSoFar = Math.max(maxSoFar, currentMax);
11        }
12        return maxSoFar;
13    }
14 }
```

Saved

Ln 15, Co

☑ Testcase ➤ Test Result

Accepted Runtime: 0 ms

☑ Case 1 ☑ Case 2 ☑ Case 3

Input

nums =
[-2,1,-3,4,-1,2,1,-5,4]

Output

6

1732. Find the Highest Altitude

Solved 

Easy Topics Companies Hint

There is a biker going on a road trip. The road trip consists of $n + 1$ points at various altitudes. The biker starts his trip on point 0 with altitude equal 0.

You are given an integer array `gain` of length `n` where `gain[i]` is the **net gain in altitude** between points `i` and `i + 1` for all ($0 \leq i < n$). Return the **highest altitude** of a point.

Example 1:

Input: `gain = [-5,1,5,0,-7]`
Output: 1
Explanation: The altitudes are `[0,-5,-4,1,1,-6]`. The highest is 1.

Example 2:

Input: `gain = [-4,-3,-2,-1,4,3,2]`
Output: 0
Explanation: The altitudes are `[0,-4,-7,-9,-10,-6,-3,-1]`. The highest is 0.

Constraints:

- `n == gain.length`

3.5K 428 ☆

```
1 class Solution {
2     public int largestAltitude(int[] gain) {
3         int maxAltitude = 0;
4         int currentAltitude = 0;
5
6         for (int g : gain) {
7             currentAltitude += g;
8             maxAltitude = Math.max(maxAltitude, currentAltitude);
9         }
10
11         return maxAltitude;
12     }
13 }
```

Saved

☒ Testcase [Test Result](#)

Accepted Runtime: 0 ms

☒ Case 1 ☒ Case 2

Input

gain =
[-5,1,5,0,-7]

Output

1

Expected

49. Group Anagrams

Medium Topics Companies

Given an array of strings `strs`, group the **anagrams** together. You can return the answer in **any order**.

Example 1:

Input: `strs = ["eat","tea","tan","ate","nat","bat"]`

Output: `[["bat"],["nat","tan"],["ate","eat","tea"]]`

Explanation:

- There is no string in `strs` that can be rearranged to form `"bat"`.
- The strings `"nat"` and `"tan"` are anagrams as they can be rearranged to form each other.
- The strings `"ate"`, `"eat"`, and `"tea"` are anagrams as they can be rearranged to form each other.

Example 2:

Input: `strs = [""]`

Output: `[[""]]`

Example 3:

Input: `strs = ["a"]`

22.4K 426

Solved

Java Auto

```
18
19
20 // Add original string to the corresponding key's list
21 map.putIfAbsent(key, new ArrayList<>());
22 map.get(key).add(str);
23
24 // Return all grouped lists
25 return new ArrayList<>(map.values());
26
27 }
```

Saved

Testcase Test Result

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

strs =
["eat","tea","tan","ate","nat","bat"]

Output

[["eat","tea","ate"],["bat"],["tan","nat"]]

Expected

347. Top K Frequent Elements

Medium Topics Companies

Given an integer array `nums` and an integer `k`, return the `k` most frequent elements. You may return the answer in any order.

Example 1:

Input: `nums = [1,1,1,2,2,3]`, `k = 2`

Output: `[1,2]`

Example 2:

Input: `nums = [1]`, `k = 1`

Output: `[1]`

Example 3:

Input: `nums = [1,2,1,2,1,2,3,1,3,2]`, `k = 2`

Output: `[1,2]`

Constraints:

19.8K 499 ☆ ↗ ⓘ

Solved ✓

Java Auto

```
1 import java.util.*;
2
3 class Solution {
4     public int[] topKFrequent(int[] nums, int k) {
5         // Step 1: Count frequency of each number
6         Map<Integer, Integer> countMap = new HashMap<>();
7         for (int num : nums) {
8             countMap.put(num, countMap.getOrDefault(num, 0) + 1);
9         }
10
11         // Step 2: Create buckets where index = frequency
```

Saved

Testcase Test Result

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

nums =
[1,1,1,2,2,3]

k =
2

Output

[Description](#) | [Editorial](#) | [Solutions](#) | [Submissions](#)

1470. Shuffle the Array

Easy | [Topics](#) | [Companies](#) | [Hint](#)

Given the array `nums` consisting of `2n` elements in the form `[x1, x2, ..., xn, y1, y2, ..., yn]`.

Return the array in the form `[x1, y1, x2, y2, ..., xn, yn]`.

Example 1:

Input: `nums = [2,5,1,3,4,7], n = 3`

Output: `[2,3,5,4,1,7]`

Explanation: Since `x1=2, x2=5, x3=1, y1=3, y2=4, y3=7` then the answer is `[2,3,5,4,1,7]`.

Example 2:

Input: `nums = [1,2,3,4,4,3,2,1], n = 4`

Output: `[1,4,2,3,3,2,4,1]`

Example 3:

Input: `nums = [1,1,2,2], n = 2`

Output: `[1,2,1,2]`

Constraints:

6.5K

166

</> Code

Java ▾ Auto

```
1 class Solution {
2     public int[] shuffle(int[] nums, int n) {
3         int[] result = new int[2 * n];
4
5         for (int i = 0; i < n; i++) {
6             result[2 * i] = nums[i];
7             result[2 * i + 1] = nums[i + n];
8         }
9
10        return result;
11    }
```

SavedLn 12, Col 2

☒ Testcase >_ Test Result

AcceptedRuntime: 0 ms

☒ Case 1☒ Case 2☒ Case 3

Input

nums =
[2,5,1,3,4,7]

n =
3

Output